

LIVELLO 0

I primi passi: ambiente, compilazione, primo programma

Dalla configurazione dell'IDE alla comprensione del ciclo di compilazione, fino all'analisi riga per riga del tuo primo programma in C.

Ambiente

Terminale

Linguaggi

6 fasi

Primo programma

Contenuti elaborati dall'autore. L'intelligenza artificiale è stata usata come supporto per impaginazione, grafica e alcuni elementi visivi.

2.1 — AMBIENTE DI SVILUPPO

Due strade per iniziare

La scelta dipende anche dal tuo sistema operativo: Windows, macOS o Linux.

CONSIGLIATO

CLion

Licenza annuale gratuita per studenti UniCa (tutti i prodotti JetBrains): registrati con la mail istituzionale ed è pronto all'uso, senza configurazioni.

LEGGERO

Visual Studio Code

Gratuito, open source e leggero. Richiede però l'estensione C/C++ e un compilatore (GCC o Clang) già installato sul computer.

Non perdere giorni a cercare l'ambiente «perfetto»: installa quello più semplice e inizia a scrivere codice.

2.2 — EDITOR DI TESTO E TERMINALE

Compilare a mano, in 4 passi

1

Scrivi il programma e salvalo con estensione .c (es. main.c)

2

Apri il terminale e spostati nella cartella con il comando `cd`

3

Compila: trasforma il testo in un file binario eseguibile

4

Lancia il programma appena generato

macOS — clang (richiamabile con gcc)

```
gcc -Wall main.c -o il_mio_programma  
./il_mio_programma
```

Manca il comando? `xcode-select --install`

Windows — GCC via MinGW-w64 (MSYS2)

```
gcc -Wall main.c -o il_mio_programma  
il_mio_programma.exe
```

In PowerShell il prefisso `.\` è obbligatorio

Il flag `-Wall` (Warning All) segnala ogni riga potenzialmente problematica, anche se il programma compila.

2.3 — LINGUAGGI DI PROGRAMMAZIONE

Tre livelli di astrazione

Oggi gli LLM scrivono codice dal linguaggio naturale

I livelli sotto restano, e conoscerli conta ancora di più.

Linguaggi di alto livello

Vicini alla logica umana e indipendenti dall'hardware.

Linguaggio Assembly

Parole chiave come ADD o MOV, ancora legate all'architettura.

Linguaggio macchina

Solo sequenze di bit (0 e 1): nativo per l'hardware, illeggibile per noi.

ASTRAZIONE

Il C è di alto livello e compilato

Compilati: tradotti tutti insieme prima dell'esecuzione

Interpretati: tradotti riga per riga durante l'esecuzione

Il codice C non è eseguibile direttamente: deve attraversare un flusso di lavoro in 6 fasi.

2.3 — LINGUAGGI DI PROGRAMMAZIONE

Dal prompt al programma in C

IL PROMPT

Scrivi un programma in C standard (C11) che gestisce una rubrica telefonica da terminale.

Ogni contatto ha nome, cognome, numero di telefono ed email. Il programma mostra un menu con queste operazioni: aggiungere un contatto, cercare per cognome (anche parziale, senza distinguere maiuscole e minuscole), eliminare un contatto, stampare tutti i contatti in ordine alfabetico per cognome, uscire.

I contatti devono stare in un array allocato dinamicamente che cresce quando serve. All'avvio il programma carica i contatti da un file di testo, all'uscita li salva nello stesso file. Gestisci gli input errati senza che il programma vada in crash e controlla il valore di ritorno di malloc, realloc e fopen.

Usa una struct, funzioni separate per ogni operazione e commenti brevi in italiano. Dopo il codice, dimmi come compilarlo con gcc.

IN POCHI SECONDI, L'LLM RESTITUISCE

Codice C11 completo

struct, una funzione per operazione, commenti in italiano

Memoria e file

array che cresce con realloc, caricamento e salvataggio su file

Controllo degli errori

input errati e ritorno di malloc, realloc e fopen

Istruzioni di compilazione

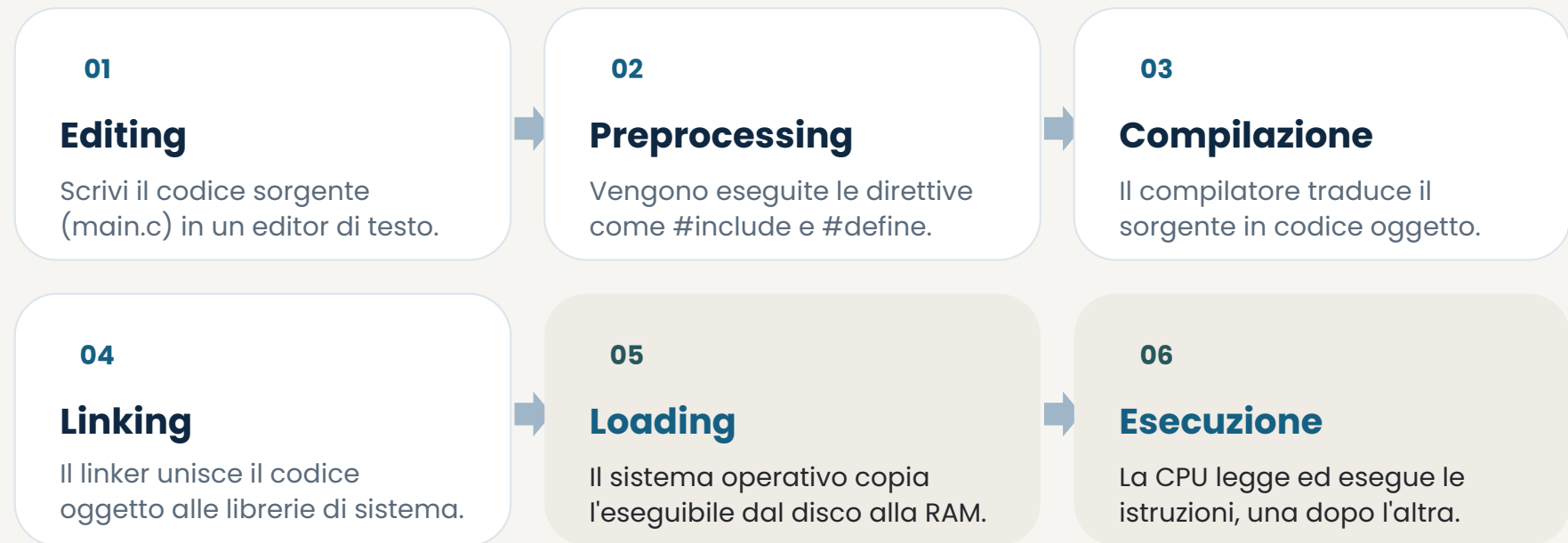
il comando gcc per ottenere l'eseguibile



Il codice generato va letto e verificato: per farlo bisogna conoscere il C.

DAL CODICE ALL'ESECUZIONE

Le 6 fasi di un programma in C



Loading ed Esecuzione sono fasi di runtime, gestite dal sistema operativo.

2.4 — LA STRUTTURA DI UN PROGRAMMA

Hello world.

PYTHON

```
print("Hello world")
```

C

```
//Il mio primo programma in C
#include <stdio.h>
int main(void) {
    printf("Hello world\n");
    return 0;
}
```

PER CAPIRLO DAVVERO SERVE CONOSCERE

<code>#include</code>	Il preprocessore e le direttive
-----------------------	---------------------------------

<code><stdio.h></code>	Librerie e file header
------------------------------	------------------------

<code>main() { }</code>	Le funzioni e la loro struttura
-------------------------	---------------------------------

<code>(void)</code>	Parametri e argomenti
---------------------	-----------------------

<code>int, return 0;</code>	Tipi di dato e valori restituiti
-----------------------------	----------------------------------

<code>printf("\n")</code>	printf e sequenze di escape
---------------------------	-----------------------------

Anche il programma più semplice tocca concetti che vedremo nelle prossime lezioni.

2.4 — LA STRUTTURA DI UN PROGRAMMA

Il tuo primo programma in C

```
//Il mio primo programma in C  
#include <stdio.h>  
int main(void) {  
    printf("Hello world\n");  
    return 0;  
}
```

OUTPUT NEL TERMINALE

Hello world

//

I commenti sono ignorati dal compilatore: usane il più possibile (anche `/* ... */`).

#include

Include un header file (stdio.h) con dichiarazioni, tra cui printf.

main

Funzione da cui parte l'esecuzione: `int` = restituisce un intero, `void` = nessun parametro.

return 0;

Comunica che tutto è andato a buon fine; nel `main`, dal C99, è implicito.

Il simbolo `\n` è un ritorno a capo. In queste slide si fa riferimento al C moderno (dal C99 in poi).